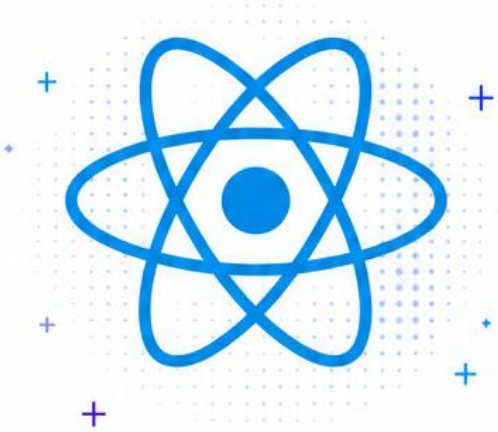
React Behind the Scenes

From User Action to Pixels on the Screen



React is not just about writing components. It has a smart architecture and an efficient rendering process that makes it incredibly fast and powerful.

In This Guide

Understand
The complete journey React takes for every update.

Explore
The core concepts like Render Phase, Commit Phase, Reconciliation, etc.

Learn
Fiber architecture and how React achieves high performance.

Visualize
The browser rendering pipeline and final output on screen.

Master
Best practices to write better, faster and more scalable React apps.

What Happens Behind the Scenes?

When a user interacts with your app (like clicking a button), a chain of smart processes runs behind the scenes to update the UI.

React doesn't update the UI directly. Instead, it creates a **Virtual DOM**, compares it with the previous one, finds the minimum changes and then updates only the necessary parts of the **Real DOM**.

This entire process happens in two major phases:

1. **Render Phase**
   React prepares the update in memory.
   It calculates what needs to be changed.

2. **Commit Phase**
   React applies those changes to the Real DOM and the browser updates the screen.

This is why React apps feel so **fast** and **smooth**!

High Level Flow

User Action
(Click, Type, Scroll, etc.)

↓

State / Props Change
State or props are updated.

↓

Render Phase (In Memory)
React creates new Virtual DOM, compares with old, finds changes.

↓

Commit Phase
React updates only the changed parts in the Real DOM.

↓

Browser Paint
Browser re-renders the changes.

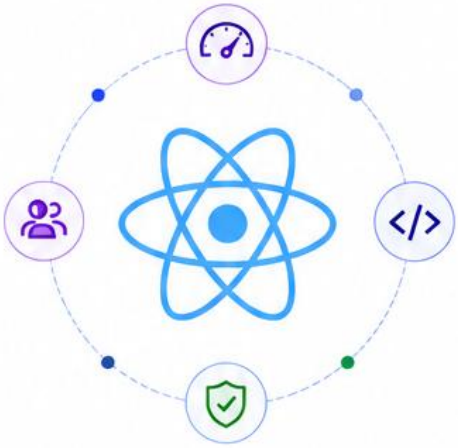
↓

Updated Screen
User sees the updated UI.

Key Takeaway

React is all about doing less work for more performance.
It updates **only what's changed**, not everything!

Why React?



Designed for a better developer & user experience.

React is more than a UI library. It is a powerful JavaScript library that helps you build fast, interactive and scalable web applications with less code and maximum performance.

Top Reasons to Use React

Blazing Fast
Virtual DOM + Efficient Diffing = Super fast updates.

Component Based
Build small, reusable pieces of UI and compose complex interfaces.

Reusability
Write once, use anywhere. Promotes cleaner and modular code.

Declarative
Describe what you want, React handles how to do it.

Strong Ecosystem
Huge community, rich libraries, tools and dev support.

Scalable
Designed to grow with your app from small to enterprise level.

The Core Idea

React follows a simple but brilliant idea:

> Instead of manipulating the Real DOM directly, React creates a **Virtual DOM** in memory, compares it with the previous one, finds the **minimum changes**, and updates **only what's necessary.**

How React Updates the UI

User Interaction → React Creates Virtual DOM → Compare & Find Changes → Update Only Changed Parts → UI Updated Efficiently

Virtual DOM vs Real DOM

Real DOM	Virtual DOM
Directly updates the actual DOM.	Updates happen in memory.
Every change causes repaint and reflow.	Only the changed parts are updated in the Real DOM.
Slower for large applications.	Faster and more efficient.
High DOM manipulation cost.	Low DOM manipulation cost.

VS

React is Perfect For

- **Single Page** Applications
- **Dynamic** Web Apps
- **Mobile Apps** (React Native)
- **Dashboards &** Admin Panels
- **Real-time** Applications

Did You Know?

Facebook, **Instagram**, **YouTube**, **Netflix**, **Airbnb** and many more **products** are built using React.

Key Takeaway

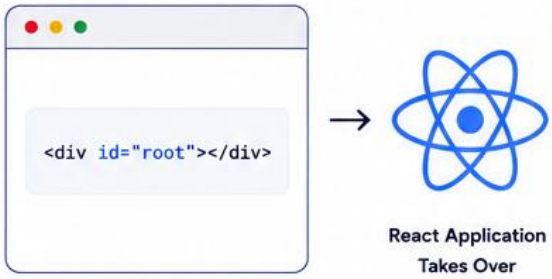
React helps developers build better UI, write less code and deliver blazing fast user experiences.

Less Code | Better Performance | Happy Users

Where React Starts

From HTML to React Application

Every React app begins its journey in the browser. Let's see how React takes control of a simple HTML page and turns it into a dynamic application.



The Starting Point: index.html

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width,
      initial-scale=1.0" />
  <title>React App</iitle>
</head>
<body>
  <div id="root"></div>  <!-- React will mount here -->
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

** **

This is a container in the HTML where React will render your entire app.

main.jsx (Entry File)

This file contains the starting point of your React application.

Browser Loads HTML First

The browser reads index.html and shows the page (but the root is empty for now).

What Happens?

1. Browser loads index.html
2. Finds the root element
3. Loads main.jsx (JavaScript)
4. React takes control and renders the app inside #root

React Takes Control

main.jsx

```
import React from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

const root = createRoot(document.getElementById('root'));

root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Let's Break It Down

`import { createRoot } from 'react-dom/client'`	Imports the modern React 18+ API used to start a React app.
`document.getElementById('root')`	Selects the `<div id="root">` from index.html.
`createRoot()`	Creates a React root. This is the entry point for your React application.
`root.render(<App />)`	Tells React to render the `<App />` component inside #root.
`<React.StrictMode>`	Helps identify potential problems in the application during development.

Visual Flow

index.html loaded
Browser loads the HTML file.

→ **#root found**
Browser finds the root container.

→ **main.jsx executed**
JavaScript bundle is loaded.

→ **React takes over**
React mounts the app inside #root.

→ **App is rendered**
UI appears on the screen.

Result in Browser



Key Takeaway

React starts from a simple HTML page and gradually takes control of the UI by rendering components inside the root element.

Pro Tip

In React 18+, always use createRoot() instead of ReactDOM.render(). It's faster and future-ready!

Update Triggered

What causes React to re-render?

An update can be triggered by many things. When that happens, React schedules work and starts the update process.

Goal

To understand what triggers an update and how React schedules it for re-render.

What Can Trigger an Update?

1. **User Interaction**
   Click, input, hover, submit, etc.
   Example: Button onClick
2. **State Update**
   Calling a state setter function (useState).
   Example: setCount(count + 1)
3. **Props Change**
   Parent component re-renders with new props.
   Example: <User name="Anshika" />
4. **Context Update**
   When a Context value changes.
   Example: ThemeContext.Provider
5. **External Store Update**
   Updates from external stores (Redux, Zustand, etc.).
   Example: store.setState()
6. **Other Cases**
   useReducer, forceUpdate (class), suspense resolution, etc.
   Example: dispatch(action)

How React Handles an Update?

1. **Update Occurs**
   Something changes (state, props, context, etc.).
2. **React Schedules the Update**
   React marks the root as having pending work. (Using Fiber & Scheduler)
3. **Work is Scheduled (Prioritized)**
   React assigns a priority to the update (e.g. high, normal, low).
4. **Render Phase Starts**
   React begins preparing the new UI in memory. (See next slide)

Important: React does NOT update the DOM at this stage. It just prepares the update in memory.

Example Code

Every time the button is clicked, an update is triggered.

```jsx
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1); // <-- Update Triggered
  }

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={handleClick}>Increment</button>
    </div>
  );
};
```

Clicking the button calls setCount(`). This triggers an update.

What Happens Behind the Scenes?

1. handleClick runs and calls setCount( ).
   This tells React: "Hey, something changed!"
2. React creates an update object with the new state.
   It puts this update into a queue.
3. React marks the fiber root as having pending work.
4. Scheduler picks this work based on priority.
   (e.g. discrete events like click = high priority)
5. When it's time, React starts the Render Phase to compute the new UI in memory.

Key Point

Update Triggered is just the beginning.
The actual DOM update happens later in Commit Phase.

Update Priorities (Simplified)

React assigns priorities so important updates happen first.

- **High (Immediate)**
  User interactions like clicks, typing, etc.
  (Needs instant response)
- **Normal**
  Most updates fall here.
  (Default priority)
- **Low**
  Non-urgent updates.
  (Can be delayed if needed)
- **Idle**
  Lowest priority.
  Runs when nothing else is happening.

Remember

React is interruptible.
If a higher priority update comes, React can pause the current work and handle the important one.

Key Takeaway

Updates are the reason React re-renders. React listens, schedules, prioritizes, and prepares — all before touching the DOM!

The Big Picture So Far

Where React Starts → Update Triggered → Render Phase → Commit Phase → UI Updated

Inside the Render Phase

React prepares the update in memory.

Once a state or props change is triggered, React enters the Render Phase.
This phase is all about **calculation, comparison** and **preparation.**

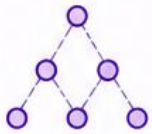
Render Phase = Preparation

- Runs in memory (no DOM updates)
- Can be paused, stopped or restarted
- May run multiple times
- Pure and side-effect free
- Finds the minimal changes needed

3 Important Steps in Render Phase

1 Create New Virtual DOM

React re-renders the component tree and creates a new Virtual DOM in memory based on the latest state/props.



2 Reconciliation (Diffing)

React compares the new Virtual DOM with the previous one (old Virtual DOM) to find the differences.



3 Prepare Changes (Effect List)

React creates a list of minimal DOM operations (add, update, delete) needed to update the Real DOM.

Effect List (Example)

- Update text of <h1>
- Add new <li> item
- Remove old <p> element

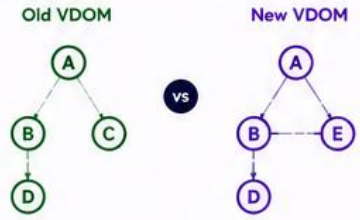
Reconciliation (Diffing) Explained

React uses a smart algorithm to efficiently compare two trees and find the minimum changes.

Key Rules of Reconciliation

1. **Different element types**
   → React replaces the entire subtree.
2. **Same element type**
   → React updates the attributes and compares children.
3. **List items should have stable "key"**
   → Helps React identify which items changed, added or removed.

Example



Changes Detected:

- C → E (Replace)
- D → D (No Change)

Example Code

Component

```
function Counter() {
  const [count, setCount] =
    useState(0);

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() =>
        setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

User Clicks Button

```
setCount(count + 1)
```

↓

```
count changes: 0 → 1
```

↓

React enters Render Phase

1. New VDOM created (count = 1)
2. Reconciliation finds the difference
3. Prepare change: Update <h1> text

New VDOM (in memory)

```
<div>
  <h1>1</h1>  ← updated
  <button>Increment</button>
</div>
```

Old VDOM (in memory)

```
<div>
  <h1>0</h1>
  <button>Increment</button>
</div>
```

What Does React Do in Render Phase?

- Builds the new Virtual DOM
- Compares with old Virtual DOM
- Finds the minimal changes
- Creates an effect list (instructions)
- Passes the effect list to the Commit Phase

Remember

Render Phase is like planning a trip. React figures out the best route (minimal changes) before performing the actual journey (DOM update).

Key Takeaway

Render Phase is all about thinking and preparing.
React calculates what should change, but doesn't touch the DOM yet.
This makes React apps fast, interruptible and user-friendly.

- **Fast** — Runs in memory
- **Interruptible** — Can be paused or aborted
- **Smart** — Updates only what's needed

Commit Phase

This is where the magic happens!

After the Render Phase prepares the changes, React enters the **Commit Phase** to apply those changes to the **Real DOM** and update the screen.

Render Phase → **Commit Phase**

Prepares Changes (In Memory)

- Create new VDOM
- Compare with old VDOM
- Find minimal changes
- Create effect list

Applies Changes (To Real DOM)

- Update Real DOM
- Browser paints
- Run useEffect callbacks
- UI is now updated!

What Happens in Commit Phase?

1. **Update the Real DOM**
   React applies all the necessary changes to the Real DOM.
2. **Browser Paint (Reflow & Repaint)**
   The browser recalculates layout (reflow), repaints pixels and updates what the user sees.
3. **Run useEffect Callbacks**
   After the DOM is updated and painted, React runs the useEffect (and useLayoutEffect) callbacks.
4. **UI is Now Updated!**
   The user sees the final updated UI on the screen.
5. **Ready for Next Update**
   React is now ready to listen for more updates and repeat the cycle.

Commit Phase Sub-steps (Detailed)

1. **Before Mutation (Snapshot)**
   React takes a snapshot (if needed) before making any changes. Used by getSnapshotBeforeUpdate.
2. **Mutation Phase**
   React updates the Real DOM with all the changes calculated in Render Phase.
3. **Layout Phase**
   React runs useLayoutEffect synchronously after DOM updates but before the browser paints.
4. **Paint (Browser)**
   Browser repaints the screen with the new changes. User sees the updated UI.
5. **Passive Effects**
   React runs useEffect asynchronously after painting is completed.

Important
useLayoutEffect runs before paint (blocking).
useEffect runs after paint (non-blocking).

Example: What Gets Updated?

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

→

Commit Phase Updates

- Updates text inside <h1> (old: 0 → new: 1)
- Nothing else changes in DOM
- Browser repaints the text
- useEffect callbacks run

Result on Screen



User sees the updated count!

Why Commit Phase is Important?

- **Data Integrity** — Ensures DOM updates happen in a controlled and predictable way.
- **Performance** — Minimizes layout thrashing and unnecessary paints.
- **User Experience** — Keeps the UI smooth and responsive for the user.
- **Lifecycle Management** — Executes effects at the right time after DOM is updated.
- **Prepares for Next Update** — After commit, React is ready for the next cycle triggered by user interactions.

Key Takeaway

Render Phase is all about calculation.
Commit Phase is all about applying those changes and showing them to the user. This separation is what makes React so fast and efficient!

Update DOM | Browser Paint | Run Effects | Updated UI

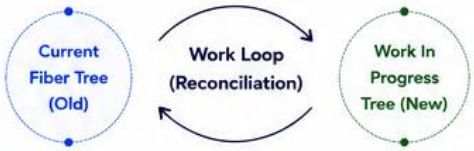
Fiber Architecture

The engine that powers React's performance.

React Fiber is the new reconciliation engine introduced in React 16. It is responsible for rendering, updating and managing the component tree in an efficient way.

What is Fiber?

Fiber is a re-implementation of the React core algorithm.
It is a virtual stack frame that represents a unit of work.



Why Fiber?

Incremental Rendering
Breaks work into small chunks and spreads it over multiple frames.

Interruptible
Can pause, resume or abort work to handle high priority updates.

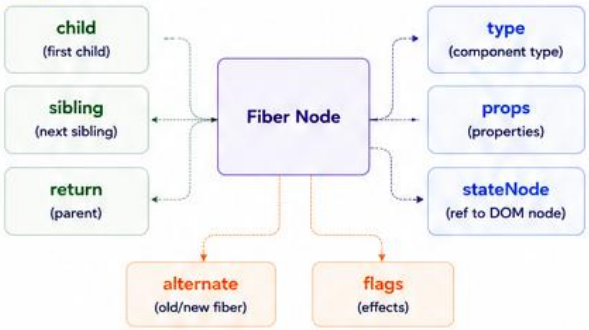
Prioritization
Updates are given priority. Important updates happen first.

Better UX
Keeps the UI responsive even during heavy updates.

Fiber vs Stack Reconciler

Stack Reconciler (Old)	Fiber Reconciler (New)
Synchronous	Asynchronous (Interruptible)
Can't pause or stop	Can pause, resume or abort
Difficult to prioritize	Supports priority-based updates
Blocks the main thread	Doesn't block the main thread
Poor user experience during heavy updates	Smooth and responsive user experience

Fiber Node Structure

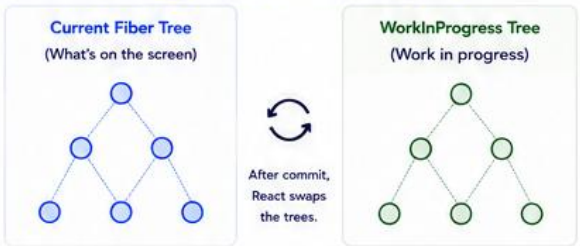


Each fiber is a lightweight JavaScript object.
These nodes form a linked tree structure.

Work Loop (How Fiber Works)

1. **Schedule Work** — React schedules an update (setState / props change / etc.).
2. **Start Work Loop** — React starts working on the fiber tree (render phase).
3. **Perform Unit of Work** — React works on one fiber node at a time.
4. **Yield (If Needed)** — If a higher priority update comes, React pauses the work.
5. **Resume Work** — React resumes from where it left off.
6. **Complete Work** — When all work is done, React moves to Commit Phase.

Double Buffering (Current & WorkInProgress)



This technique is called "Double Buffering".
It ensures the UI is updated smoothly without losing the current screen.

Key Benefits of Fiber

Non-blocking Rendering
Doesn't block the main thread.

Prioritized Updates
Important updates happen first.

Better for Animations
Smooth 60fps experience.

Scalable Applications
Handles large apps with ease.

Future Ready
Enables features like Concurrent Mode, Suspense, Transitions etc.

Fiber in Action (Example)



Key Takeaway

Fiber is the secret behind React's speed and smooth user experience.
It breaks work into small units, prioritizes updates and keeps the UI responsive even in large and complex applications.

Remember

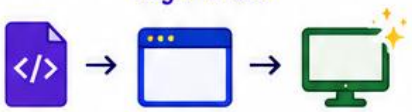
- Fiber is not a feature. It's the internal architecture.
- It makes React fast, interruptible and future-ready.
- It works quietly behind the scenes for you!

Browser Rendering Pipeline

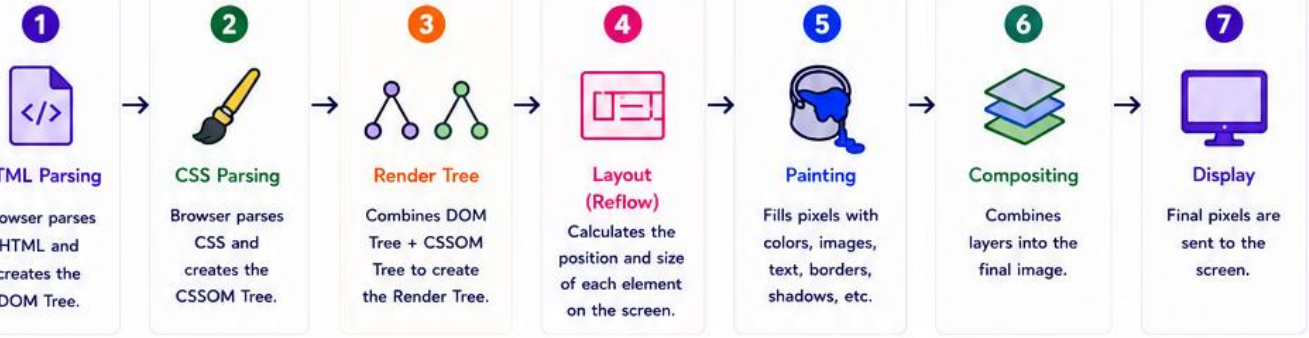
From Code to Pixels on Screen!

After React updates the DOM, the browser takes over and turns it into what we see on the screen.

Big Picture



The 7 Steps of Browser Rendering Pipeline

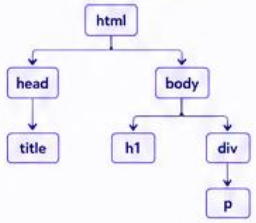


React updates the DOM → Browser follows this pipeline → You see the updated UI!

1 HTML Parsing

- Browser reads HTML line by line.
- Creates the DOM Tree (Document Object Model).

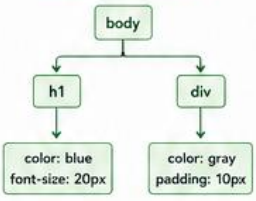
DOM Tree (Example)



2 CSS Parsing

- Browser reads CSS rules.
- Creates the CSSOM Tree (CSS Object Model).

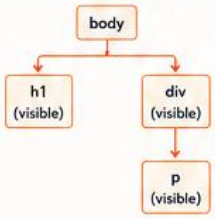
CSSOM Tree (Example)



3 Render Tree

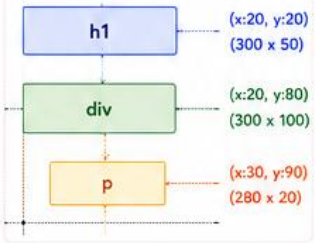
- Combines DOM Tree + CSSOM Tree.
- Only visible elements are included.

Render Tree (Example)



4 Layout (Reflow)

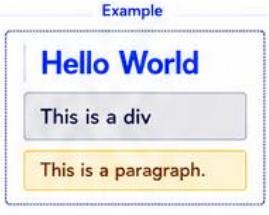
- Calculates exact position and size of each element.
- Determines the geometry.



5 Painting

- Fills each pixel with the right color, text, images, borders, etc.
- Produces paint commands.

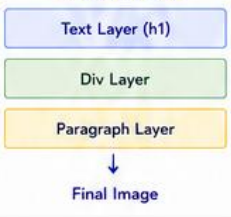
Example



6 Compositing

- Browser creates layers (for elements).
- Combines layers into the final image.

Layers (Example)



7 Display

- Final image is sent to GPU.
- Pixels are drawn on the screen.

Output



Key Takeaway

- React updates the DOM.
- Browser converts it into pixels.
- Many steps happen super fast.
- This is why the web feels fast!

Important Notes

- Reflow (Layout) is expensive. Avoid triggering it frequently.
- Use transforms and opacity for smooth animations.
- Modern browsers optimizes these steps heavily.
- This pipeline runs every time something changes on the page.

Common Triggers That Cause Re-render

- Changing style (width, height, margin, etc.)
- Adding/removing elements
- Changing class or id
- Window resize
- Reading layout properties (offsetWidth, etc.)

In One Line

Code → DOM → Render Tree → Layout → Paint → Composite → Display = Pixels on Screen

Key Takeaway

Fiber is the secret behind React's speed and smooth user experience. It breaks work into small units, prioritizes updates and keeps the UI responsive even in large and complex applications.

Remember

React updates the DOM. Browser makes it visible.



The Complete React Flow

Putting it all together!

Let's see the complete journey from a user action to the updated UI on the screen.

Goal

To understand the big picture. How does React take an update request and turn it into pixels on the screen?

End-to-End React Flow

1. **User Interaction** — User clicks a button, types in input, or performs any action in the app.
2. **Update Triggered** — React event handler updates state or props using setState, useState or parent re-render.
3. **Render Phase (Reconciliation)** — React creates new Virtual DOM, compares with old one (diffing) and prepares the minimal changes.
4. **Commit Phase** — React applies the changes to the Real DOM.
5. **Browser Paint** — Browser calculates layout, repaints the pixels and updates what the user sees.
6. **Effects Run** — useLayoutEffect runs first (after DOM update, before paint). Then useEffect runs (after paint).
7. **Ready for Next Update** — React is now ready to listen for more updates and repeat the cycle.

Trigger | Render Phase (In Memory) | Commit Phase (Real DOM Update) | Browser Work | Effects | Idle / Next Cycle

Render Phase (In Memory)

- Creates new Virtual DOM.
- Compares with old Virtual DOM.
- Finds the minimum changes (Diffing).
- Prepares Effect List (what to update in DOM).
- Does NOT touch the Real DOM.

Fast, interruptible, and happens in memory.

Commit Phase (Real DOM Update)

- Applies all the changes to the Real DOM.
- Browser updates the UI.
- Runs useLayoutEffect (before paint).
- Browser paints the screen.
- Runs useEffect (after paint).

Synchronous, non-interruptible.

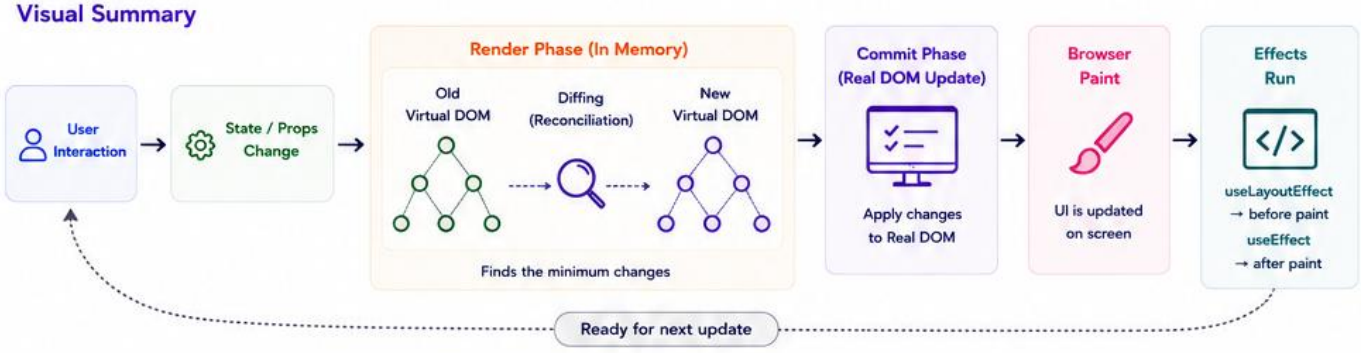
Key Takeaway

React is smart because it splits the work into two parts:

1. Render Phase → Think & Prepare (can be paused or restarted)
2. Commit Phase → Apply & Show (fast and non-blocking)

This separation makes React fast, responsive and efficient.

Visual Summary



Quick Reference

Trigger	User action → state/props change
Render Phase	Calculate (in memory), can be paused
Commit Phase	Apply changes (real DOM update)
Browser Paint	Screen is updated
Effects	useLayoutEffect → before paint useEffect → after paint
Repeat	React is ready for next update

Common Myths vs Reality

Myth: React updates the DOM directly.	**Reality:** React uses Virtual DOM and updates only what's needed.
Myth: useEffect runs before paint.	**Reality:** useEffect runs **after** paint.
Myth: Render Phase is synchronous.	**Reality:** Render Phase is interruptible in React 18+.

Remember This!

- React is not magic. It's a well-designed system.
- Fiber is the engine behind it.
- Virtual DOM is just a JavaScript representation.
- Separation of Render & Commit is the secret of performance.

You're doing great!

You now know how React works behind the scenes. Keep building, keep learning, keep improving!

Understand | Practice | Build | Repeat

React Behind the Scenes Interview Cheat Sheet

Quick revision of everything we learned!

A one-page guide to help you remember the core internals of React.

> React makes your app fast by doing the hard work behind the scenes.
>
> It thinks, prepares and then updates — in the smartest way possible.

Key Concepts at a Glance

Virtual DOM	A lightweight JavaScript representation of the Real DOM.
Reconciliation	The process of comparing old and new Virtual DOM to find minimum changes.
Diffing	Algorithm used in reconciliation to detect what changed (added, updated, removed).
Render Phase	React prepares the update in memory. Does NOT update the Real DOM.
Commit Phase	React applies the changes to the Real DOM, updates the UI and runs effects.
Fiber	React's internal engine that enables incremental, interruptible and prioritized rendering.

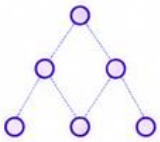
React Flow (Quick Recap)

1. **Update Triggered** — State/Props change or user interaction
2. **Render Phase (In Memory)**
   - Create new VDOM
   - Reconciliation
   - Diffing
   - Prepare effect list
3. **Commit Phase (Real DOM Update)**
   - Update DOM
   - Browser Paint
   - useLayoutEffect
   - useEffect
4. **UI Updated** — User sees the changes

Render Phase = Think & Prepare (can be paused)

Commit Phase = Apply & Show (cannot be paused)

Fiber Architecture in Short



- Fiber is a virtual stack frame (a unit of work).
- It forms a linked list (tree) structure.
- Work is divided into small units.
- React can pause, resume or abort work.
- This makes React fast and responsive.

Browser Rendering Pipeline (Mini Recap)

1. **HTML** — Parse HTML
2. **CSS** — Parse CSS
3. **Render Tree** — Build Render Tree
4. **Layout** — Calculate Layout
5. **Paint** — Paint on Screen

React updates the DOM → Browser follows this pipeline → UI appears on screen.

Phase Comparison

Render Phase	Commit Phase
Runs in memory	Runs on Real DOM
Can be paused, aborted or restarted	Cannot be paused
May run multiple times	Runs once per update
Finds minimum changes	Applies changes
Pure and side-effect free	Can have side effects
Interruptible	Synchronous (Non-interruptible)

Common Interview Questions

Q1. What is the difference between Render Phase and Commit Phase?
Render Phase prepares updates in memory. Commit Phase applies them to the Real DOM.

Q2. What is Virtual DOM?
It is a JS representation of the Real DOM. React uses it to optimize UI updates.

Q3. What is Reconciliation?
It is the process of comparing old and new Virtual DOM to find the differences.

Q4. What is Fiber?
Fiber is React's internal implementation of its rendering engine.

Q5. Why is React fast?
Because of Fiber, it can break work into small units, prioritize updates and update only what's needed.

Q6. What are useLayoutEffect and useEffect?
useLayoutEffect runs before paint. useEffect runs after paint.

Key Benefits of React Internals

- **Performance** — Updates only what's needed.
- **Interruptible** — Can pause, resume or abort work.
- **Priority Based** — Important updates happen first.
- **Responsive UI** — Smooth experience even in heavy apps.
- **Reliable** — Predictable and consistent updates.

Common Mistakes (Avoid These!)

✗ **Wrong:** Fiber is Virtual DOM.	→ **Correct:** Fiber is the engine, VDOM is the result.
✗ **Wrong:** React updates the DOM directly.	→ **Correct:** React updates VDOM first, then Real DOM.
✗ **Wrong:** useEffect runs before paint.	→ **Correct:** useEffect runs after paint.
✗ **Wrong:** Render Phase is slow.	→ **Correct:** Commit Phase (DOM update) is quick.
✗ **Wrong:** Reconciliation compares DOM.	→ **Correct:** Reconciliation compares Virtual DOM.

Life of an Update (Story in One Line)

User does something → State/Props change → Render Phase (prepare in memory) → Commit Phase (apply to DOM) → Browser Paint → **Effects** run → **UI updated**

Golden Rule
React does the heavy thinking in memory and touches the DOM only when necessary.

Remember These 5 Words

- **Think** (Render Phase)
- **Prepare** (VDOM + Diffing)
- **Apply** (Commit Phase)
- **Paint** (Browser Work)
- **Repeat** (Next Update)

You've made it!
You now understand how React works behind the scenes.
Keep practicing, keep building, keep growing! 🚀

Code daily · Read docs · Build projects · Teach others · Keep improving